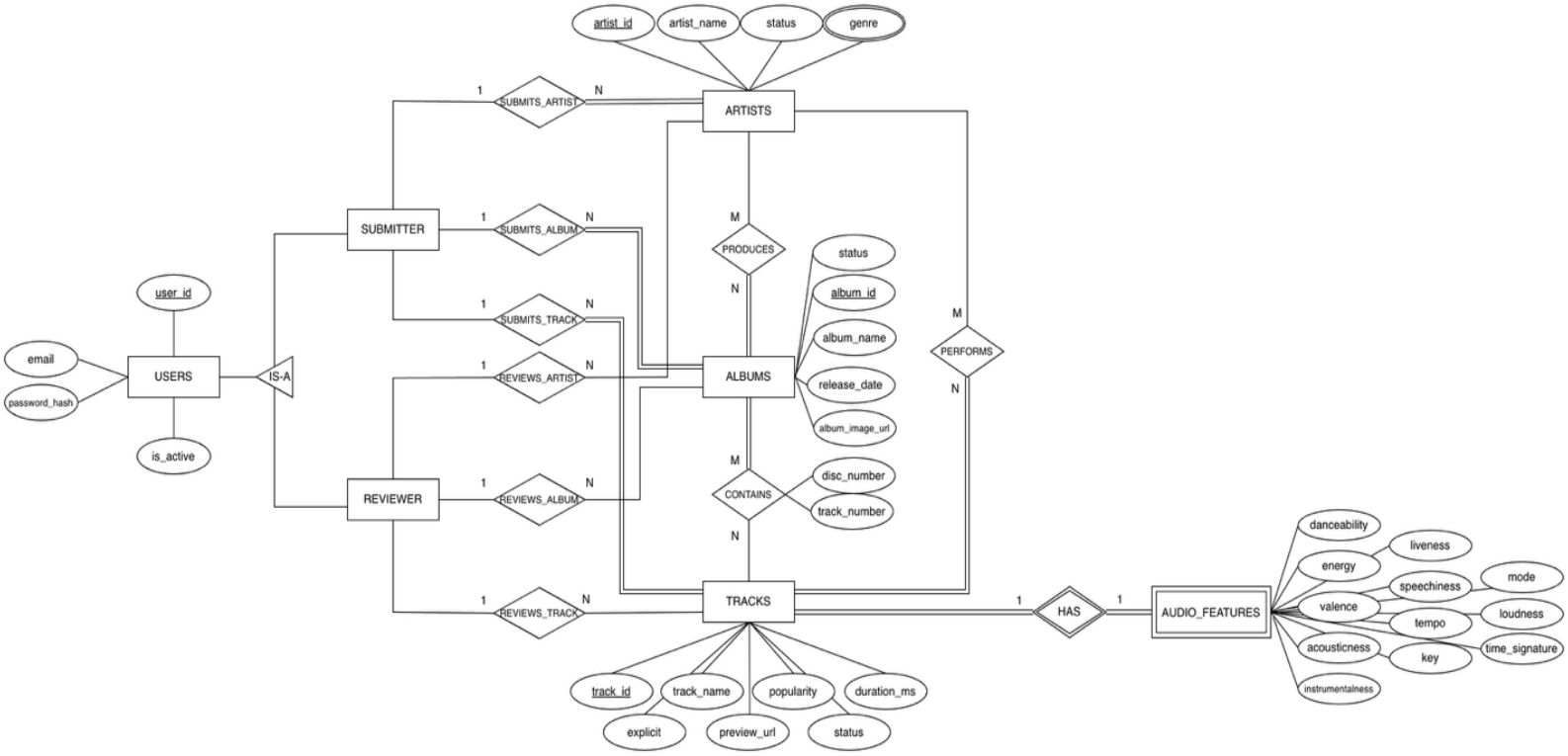
DSCC461 Database System
Project – Milestone 2

Project Name: MusicBox: A Relational Database System for Historical Music Data and Audio Feature Analysis
Team Number: 6
Member 1: Muhui Wang (mwang116)
Member 2: Sherry Wang (ywang506)

Task A: ER Diagram

1. ER Diagram



There is a weak entity in our ER diagram: **AUDIO_FEATURES**. It depends on **TRACKS** and is identified through the **HAS** relationship. AUDIO_FEATURES does not require a partial key because it has a one-to-one identifying relationship with TRACKS, and the primary key of TRACKS (track_id) is sufficient to uniquely identify each AUDIO_FEATURES record.

There is also a class hierarchy using specialization where **USERS** is the superclass and **SUBMITTER** and **REVIEWER** are subclasses connected through an IS-A relationship.

2. Cardinality and Participation:

There are ten relationships in our ER diagram.

Group 1 – Submission relationships

1. **SUBMITTER and ARTISTS – SUBMITS_ARTIST:** SUBMITTER and ARTISTS have a **1:N binary relationship** since one SUBMITTER can submit multiple artists to the system, but each artist is submitted by only one submitter. ARTISTS has **total participation** in SUBMITS_ARTIST because every artist must be submitted by exactly one submitter, while SUBMITTER has **partial participation** because a submitter may exist without submitting any artists.
   (min, max): SUBMITTER (0, N), ARTISTS (1, 1).
2. **SUBMITTER and ALBUMS – SUBMITS_ALBUM:** SUBMITTER and ALBUMS have a **1:N binary relationship** since one SUBMITTER can submit multiple albums, but each album is submitted by only one submitter. ALBUMS has **total participation** in SUBMITS_ALBUM because every album must be submitted by exactly one submitter, while SUBMITTER has **partial participation** because a submitter may exist without submitting any albums.
   (min, max): SUBMITTER: (0, N),ALBUMS: (1, 1)
3. **SUBMITTER and TRACKS – SUBMITS_TRACK:** SUBMITTER and TRACKS have a **1:N binary relationship** since one SUBMITTER can submit multiple tracks, but each track is submitted by only one submitter. TRACKS has **total participation** in SUBMITS_TRACK because every track must be submitted by exactly one submitter, while SUBMITTER has **partial participation** because a submitter may exist without submitting any tracks.
   (min, max): SUBMITTER: (0, N), TRACKS: (1, 1)

Group 2 – Review relationships

4. **REVIEWER and ARTISTS – REVIEWS_ARTIST:** REVIEWER and ARTISTS have a **1:N binary relationship** since one REVIEWER can review multiple artists, but each artist is reviewed by only one reviewer. ARTISTS has **partial participation** in REVIEWS_ARTIST because some artists may not have been reviewed yet. REVIEWER also has **partial participation** because a reviewer may exist without reviewing any artists.
   (min, max): REVIEWER: (0, N), ARTISTS: (0, 1)
5. **REVIEWER and ALBUMS – REVIEWS_ALBUM:** REVIEWER and ALBUMS have a **1:N binary relationship** since one REVIEWER can review multiple albums, but each album is reviewed by only one reviewer. ALBUMS has **partial participation** in REVIEWS_ALBUM because some albums may not have been reviewed yet. REVIEWER also has **partial participation** because a reviewer may exist without reviewing any albums.
   (min, max): REVIEWER: (0, N), ALBUMS: (0, 1)
6. **REVIEWER and TRACKS – REVIEWS_TRACK:** REVIEWER and TRACKS have a **1:N binary relationship** since one REVIEWER can review multiple tracks, but each track is reviewed by only one reviewer. TRACKS has **partial participation** in REVIEWS_TRACK because some tracks may not have been reviewed yet. REVIEWER

also has **partial participation** because a reviewer may exist without reviewing any tracks.
(min, max): REVIEWER: (0, N), TRACKS: (0, 1)

Group 3 – Music relationships

7. **ARTISTS and ALBUMS – PRODUCES:** ARTISTS and ALBUMS have a **M:N binary relationship** since one artist can produce multiple albums and one album can be produced by multiple artists. ARTISTS has **partial participation** in PRODUCES because an artist may exist before being linked to any album, while ALBUMS has **total participation** in PRODUCES because every album must be produced by at least one artist.
(min, max): ARTISTS: (0, N), ALBUMS: (1, N)
8. **ALBUMS and TRACKS – CONTAINS:** ALBUMS and TRACKS have a **M:N binary relationship** since one album contains multiple tracks and a track may appear in multiple albums (for example, compilation albums). This relationship has attributes **disc_number** and **track_number**, which describe the order of tracks in the album. ALBUMS has **total participation** in CONTAINS because every album must contain at least one track, while TRACKS has **partial participation** because a track may exist independently (e.g., as a single release) without being associated with any album.
(min, max): ALBUMS: (1, N), TRACKS: (0, N)
9. **ARTISTS and TRACKS – PERFORMS**: ARTISTS and TRACKS have a **M:N binary relationship** since one artist can perform multiple tracks and one track can be performed by multiple artists. TRACKS has **total participation** in PERFORMS because every track must be performed by at least one artist, while ARTISTS has **partial participation** because an artist may exist before being associated with any track.
(min, max): ARTISTS: (0, N), TRACKS: (1, N)

Group 4 – Audio feature relationship

10. **TRACKS and AUDIO_FEATURES – HAS:** TRACKS and AUDIO_FEATURES have a **1:1 binary relationship** since each audio feature record corresponds to exactly one track, and each track must have a corresponding audio feature record. Both TRACKS and AUDIO_FEATURES have **total participation** in this relationship because each track must have exactly one corresponding audio feature record, and each audio feature record must correspond to exactly one track.
(min, max): TRACKS: (1, 1), AUDIO_FEATURES: (1, 1)

Task B: Relational Database Design Using ER-to-Relational Mapping

1. ER-to-Relational Mapping Algorithm

Step 1: Mapping of Regular Entity Types

For each strong entity type in the ER diagram, we create a corresponding relation.

- **USERS**: Created with attributes user_id (PK), email, password_hash, role, and is_active.
- **ARTISTS**: Created with attributes artist_id (PK), artist_name, and status.
- **ALBUMS**: Created with attributes album_id (PK), album_name, release_date, album_image_url, and status.
- **TRACKS**: Created with attributes track_id (PK), track_name, popularity, duration_ms, explicit, preview_url, and status.

Step 2: Mapping of Weak Entity Types

The **AUDIO_FEATURES** entity is modeled as a weak entity (double-bordered rectangle) depending on its owner entity **TRACKS**.

- A relation **Audio_Features** is created.
- The primary key is track_id, which also serves as a foreign key referencing TRACKS(track_id).
- All specific audio metrics (danceability, energy, etc.) are included as attributes.

Step 3: Mapping of Binary 1:1 Relationship Types

The 1:1 relationship between **TRACKS** and **AUDIO_FEATURES** is already handled in Step 2 via the weak entity mapping. No additional relations are needed.

Step 4: Mapping of Binary 1:N Relationship Types

The submission and review actions are 1:N relationships between **USERS** and the content entities (**ARTISTS**, **ALBUMS**, **TRACKS**).

- Foreign keys submitted_by and reviewed_by are added to the **ARTISTS**, **ALBUMS**, and **TRACKS** relations, all referencing USERS(user_id).

Step 5: Mapping of Binary M:N Relationship Types

The ER diagram identifies several many-to-many relationships, which are mapped into separate junction tables:

1. **ARTISTS — PERFORMS — TRACKS**: Mapped to **TRACK_ARTISTS** with PK (track_id, artist_id).
2. **ALBUMS — CONTAINS — TRACKS**: Mapped to **ALBUM_TRACKS** with PK (album_id, track_id). Relationship attributes disc_number and track_number are included here.
3. **ARTISTS — PRODUCES — ALBUMS**: Mapped to **ALBUM_ARTISTS** with PK (album_id, artist_id).

Step 6: Mapping of Multivalued Attributes

The **genre** attribute on the **ARTISTS** entity is multivalued (double-oval).

- A separate relation **ARTIST_GENRES** is created.
- It contains artist_id (FK) and genre.
- The primary key is the combination (artist_id, genre).

Step 7: Mapping of N-ary Relationship Types

Not applicable. The design only consists of binary relationships.

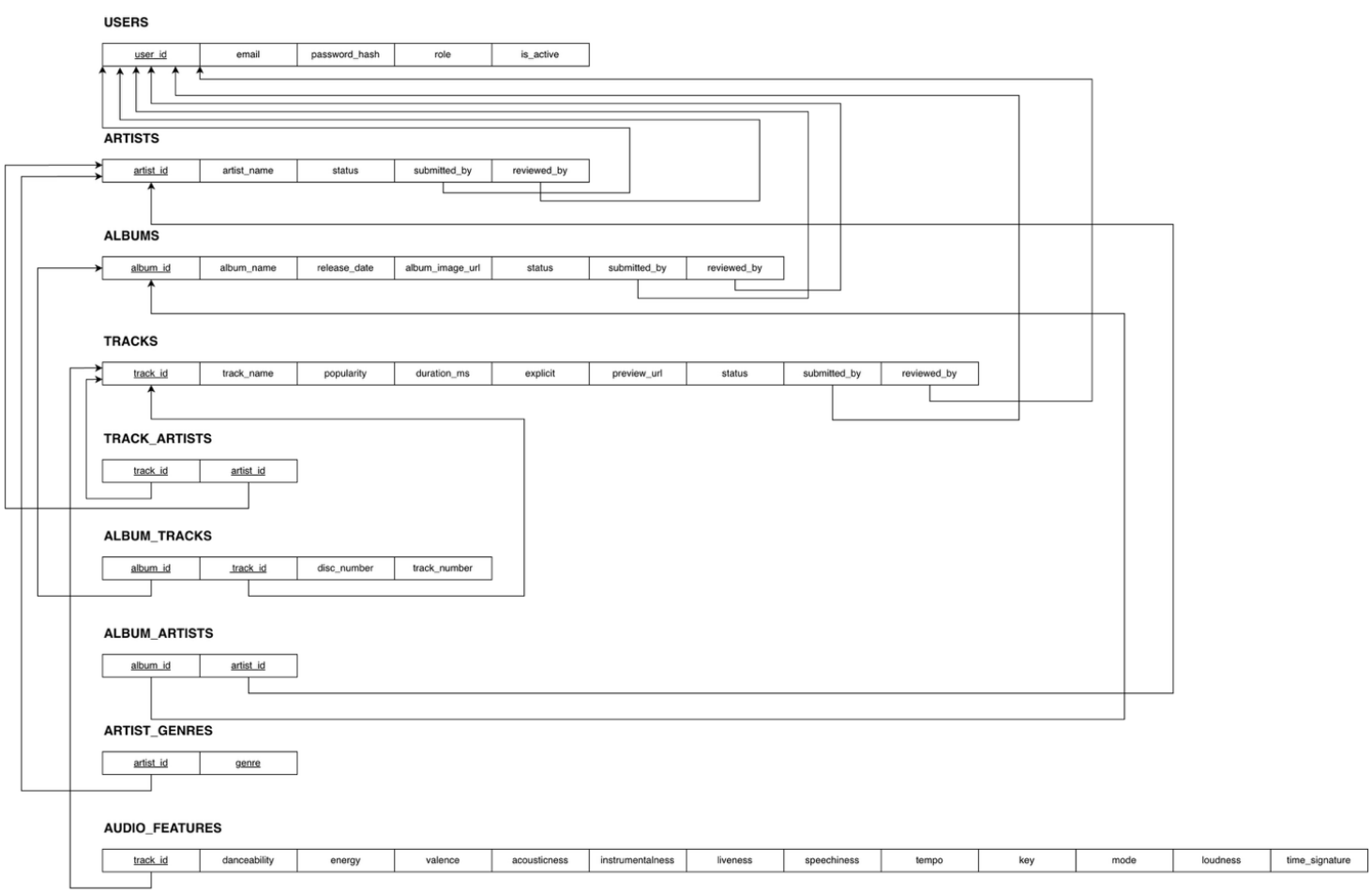
Step 8: Options for Mapping Specialization or Generalization

The ER diagram shows a specialization where **USERS** is the superclass of **SUBMITTER** and **REVIEWER** (indicated by the **IS-A** triangle).

- **Mapped using Option 8C (Single relation with one type attribute)**:
  - Instead of creating separate tables for subclasses, a single **Users** table is used.
  - A **role** attribute is included as the discriminating attribute to indicate whether a user is a SUBMITTER, a REVIEWER.
  - This is the most efficient choice since the sub-classes are disjoint (SUBMITTER can only submit and REVIEWER can only review) and do not have unique local attributes in this specific design.

Step 9: Mapping of Union Types (Categories)

Not applicable. The design does not include union types (categories).



2. Summary Table of Mapping

Relation Name	ER Diagram Components
USERS	E(USERS)
ARTISTS	E(ARTISTS) + R(SUBMITS_ARTIST) + R(REVIEWS_ARTIST)
ALBUMS	E(ALBUMS) + R(SUBMITS_ALBUM) + R(REVIEWS_ALBUM)
TRACKS	E(TRACKS) + R(SUBMITS_TRACK) + R(REVIEWS_TRACK)
TRACK_ARTISTS	R(PERFORMS)
ALBUM_TRACKS	R(CONTAINS)
ALBUM_ARTISTS	R(PRODUCES)
ARTIST_GENRES	A(GENRE)
AUDIO_FEATURES	E(AUDIO_FEATURES) + R(HAS)

3. Database Schema

This section presents the final relational schema of the MusicBox database. For each relation, we specify the primary key, foreign keys, unique keys (if any), attribute descriptions, nullability, default values, and the action taken when a referenced tuple is deleted.

3.1 List of Relations

The final database consists of the following relations:

1. USERS
2. ARTISTS
3. ALBUMS
4. TRACKS
5. TRACK_ARTISTS
6. ALBUM_TRACKS
7. ALBUM_ARTISTS
8. ARTIST_GENRES
9. AUDIO_FEATURES

3.2 Relation-by-Relation Schema Description

- USERS

Schema

USERS(user_id, email, password_hash, role, is_active)

Keys

- Primary Key: user_id
- Unique Keys: email
- Foreign Keys: None

Purpose

Stores authenticated users of the system. In the conceptual model, users may play roles such as submitter or reviewer. In the final relational schema, these responsibilities are implemented through a single **USERS** relation together with the **role** attribute and workflow-related foreign keys in other relations.

Attribute Description

Attribute	Data Type	Null	Default	Description
user_id	BIGINT	No	—	Unique identifier for each user
email	VARCHAR(255)	No	—	Email address used for login; must be unique
password_hash	VARCHAR(255)	No	—	Hashed password for authentication
role	VARCHAR(20)	No	—	User role used for authorization and workflow responsibilities
is_active	BOOLEAN	No	TRUE	Indicates whether the user account is active

Foreign Key Delete Actions

None, because **USERS** does not reference any other relation.

- ARTISTS

Schema

ARTISTS(artist_id, artist_name, status, submitted_by, reviewed_by)

Keys

- Primary Key: artist_id
- Foreign Keys:
  - submitted_by → USERS(user_id)
  - reviewed_by → USERS(user_id)
- Unique Keys: None

Purpose

Stores artist records managed by the system. Submission and review information is tracked through references to the **USERS** relation.

Attribute Description

Attribute	Data Type	Null	Default	Description
artist_id	VARCHAR(64)	No	—	Unique identifier for each artist
artist_name	VARCHAR(300)	No	—	Name of the artist
status	VARCHAR(20)	No	'pending'	Current workflow status of the artist record (e.g., pending, approved, rejected)
submitted_by	BIGINT	No	-	User who submitted the artist record
reviewed_by	BIGINT	Yes	NULL	User who reviewed the artist record

Foreign Key Delete Actions

- submitted_by: ON DELETE RESTRICT
- reviewed_by: ON DELETE SET NULL

Explanation

If a referenced user who submitted an artist record is deleted, the deletion is restricted to preserve data integrity.

If a reviewer is deleted, the artist record remains in the database, and the reviewed_by reference is set to NULL.

- ALBUMS

Schema

ALBUMS(album_id, album_name, release_date, album_image_url, status, submitted_by, reviewed_by)

Keys

- Primary Key: album_id
- Foreign Keys:
  - submitted_by → USERS(user_id)
  - reviewed_by → USERS(user_id)
- Unique Keys: None

Purpose

Stores album records in the system, including album metadata and workflow tracking information.

Attribute Description

Attribute	Data Type	Null	Default	Description
album_id	VARCHAR(64)	No	—	Unique identifier for each album
album_name	VARCHAR(400)	No	—	Name of the album
release_date	DATE	Yes	NULL	Album release date
album_image_url	TEXT	Yes	NULL	URL of the album cover image
status	VARCHAR(20)	No	'pending'	Workflow status of the album record (e.g., pending, approved, rejected)
submitted_by	BIGINT	No	-	User who submitted the album record
reviewed_by	BIGINT	Yes	NULL	User who reviewed the album record

Foreign Key Delete Actions

- submitted_by: ON DELETE RESTRICT
- reviewed_by: ON DELETE SET NULL

Explanation

If a referenced submitter is deleted, the deletion is restricted to preserve data integrity.
If a referenced reviewer is deleted, the album record remains in the database, and the reviewed_by foreign key is set to NULL.

- TRACKS

Schema

TRACKS(track_id, track_name, popularity, duration_ms, explicit, preview_url, status, submitted_by, reviewed_by)

Keys

- Primary Key: track_id
- Foreign Keys:
  - submitted_by → USERS(user_id)
  - reviewed_by → USERS(user_id)
- Unique Keys: None

Purpose

Stores track records and related workflow metadata.

Attribute Description

Attribute	Data Type	Null	Default	Description
track_id	VARCHAR(64)	No	—	Unique identifier for each track
track_name	VARCHAR(400)	No	—	Name of the track
popularity	SMALLINT	Yes	NULL	Popularity score of the track
duration_ms	INT	Yes	NULL	Duration of the track in milliseconds
explicit	BOOLEAN	Yes	NULL	Indicates whether the track contains explicit content
preview_url	TEXT	Yes	NULL	URL for the track preview audio
status	VARCHAR(20)	No	'pending'	Workflow status of the track record (e.g., pending, approved, rejected)
submitted_by	BIGINT	No	-	User who submitted the track record
reviewed_by	BIGINT	Yes	NULL	User who reviewed the track record

Foreign Key Delete Actions

- submitted_by: ON DELETE RESTRICT
- reviewed_by: ON DELETE SET NULL

Explanation

If a referenced submitter is deleted, the deletion is restricted to preserve data integrity.

If a referenced reviewer is deleted, the track record remains in the database, and the reviewed_by foreign key is set to NULL.

- TRACK_ARTISTS

Schema

TRACK_ARTISTS(track_id, artist_id)

Keys

- Primary Key: (track_id, artist_id)
- Foreign Keys:
  - track_id → TRACKS(track_id)
  - artist_id → ARTISTS(artist_id)
- Unique Keys: None

Purpose

Resolves the many-to-many relationship between tracks and artists.

Attribute Description

Attribute	Data Type	Null	Default	Description
track_id	VARCHAR(64)	No	—	Referenced track identifier
artist_id	VARCHAR(64)	No	—	Referenced artist identifier

Foreign Key Delete Actions

- track_id: ON DELETE CASCADE
- artist_id: ON DELETE CASCADE

Explanation

This relation only stores associations between tracks and artists. If either parent record is deleted, the corresponding association should also be removed automatically.

- ALBUM_TRACKS

Schema

ALBUM_TRACKS(album_id, track_id, disc_number, track_number)

Keys

- Primary Key: (album_id, track_id)
- Foreign Keys:
  - album_id → ALBUMS(album_id)
  - track_id → TRACKS(track_id)
- Unique Keys: None

Purpose

Resolves the many-to-many relationship between albums and tracks. The attributes **disc_number** and **track_number** describe the position of a track within a specific album.

Attribute Description

Attribute	Data Type	Null	Default	Description
album_id	VARCHAR(64)	No	—	Referenced album
track_id	VARCHAR(64)	No	—	Referenced track

disc_number	SMALLINT	Yes	NULL	Disc number of the track within the album
track_number	SMALLINT	Yes	NULL	Track number within the disc/album

Foreign Key Delete Actions

- album_id: ON DELETE CASCADE
- track_id: ON DELETE CASCADE

Explanation

This table represents album-track associations and has no independent meaning without its parent records. If either the album or the track is deleted, the association should be deleted automatically.

- ALBUM_ARTISTS

Schema

ALBUM_ARTISTS(album_id, artist_id)

Keys

- Primary Key: (album_id, artist_id)
- Foreign Keys:
  - album_id → ALBUMS(album_id)
  - artist_id → ARTISTS(artist_id)
- Unique Keys: None

Purpose

Resolves the many-to-many relationship between albums and artists.

Attribute Description

Attribute	Data Type	Null	Default	Description
album_id	VARCHAR(64)	No	—	Referenced album
artist_id	VARCHAR(64)	No	—	Referenced artist

Foreign Key Delete Actions

- album_id: ON DELETE CASCADE
- artist_id: ON DELETE CASCADE

Explanation

If either the album or the artist is deleted, the relationship record should also be removed automatically.

- ARTIST_GENRES

Schema

ARTIST_GENRES(artist_id, genre)

Keys

- Primary Key: (artist_id, genre)
- Foreign Keys: artist_id → ARTISTS(artist_id)
- Unique Keys: None

Purpose

Stores the multivalued attribute **genre** of **ARTIST**. Each row represents one genre value associated with one artist.

Attribute Description

Attribute	Data Type	Null	Default	Description
artist_id	VARCHAR(64)	No	—	Referenced artist
genre	VARCHAR(120)	No	—	One genre associated with the artist

Foreign Key Delete Actions

- artist_id: ON DELETE CASCADE

Explanation

If an artist is deleted, the associated genre entries should also be removed. **genre** is not a foreign key because the design does not use a separate **GENRES** relation.

- AUDIO_FEATURES

Schema

AUDIO_FEATURES(track_id, danceability, energy, valence, acousticness, instrumentalness, liveness, speechiness, tempo, key, mode, loudness, time_signature)

Keys

- Primary Key: track_id
- Foreign Keys: track_id → TRACKS(track_id)
- Unique Keys: None

Purpose

Stores numerical audio features for a track. This relation implements the 1:1 relationship between **TRACKS** and **AUDIO_FEATURES**.

Attribute Description

Attribute	Data Type	Null	Default	Description
track_id	VARCHAR(64)	No	—	Referenced track; also the primary key
danceability	DECIMAL(5,4)	Yes	NULL	Danceability score
energy	DECIMAL(5,4)	Yes	NULL	Energy score
valence	DECIMAL(5,4)	Yes	NULL	Valence score
acousticness	DECIMAL(7,6)	Yes	NULL	Acousticness score
instrumentalness	DECIMAL(10,9)	Yes	NULL	Instrumentalness score
liveness	DECIMAL(5,4)	Yes	NULL	Liveness score
speechiness	DECIMAL(5,4)	Yes	NULL	Speechiness score
tempo	DECIMAL(7,3)	Yes	NULL	Tempo in beats per minute
key	SMALLINT	Yes	NULL	Estimated musical key
mode	SMALLINT	Yes	NULL	Modality of the track
loudness	DECIMAL(7,3)	Yes	NULL	Overall loudness in decibels
time_signature	SMALLINT	Yes	NULL	Estimated time signature

Foreign Key Delete Actions

- track_id: ON DELETE CASCADE

Explanation

Audio features depend entirely on the referenced track. If the track is deleted, the corresponding audio feature record should also be deleted.

Group Work & Contribution
We all worked together on online sharing documents for the entire milestone.
We all agree each of us contributed an equal amount of work to this assignment.

Thus, each of us accomplished approximately half of the work.